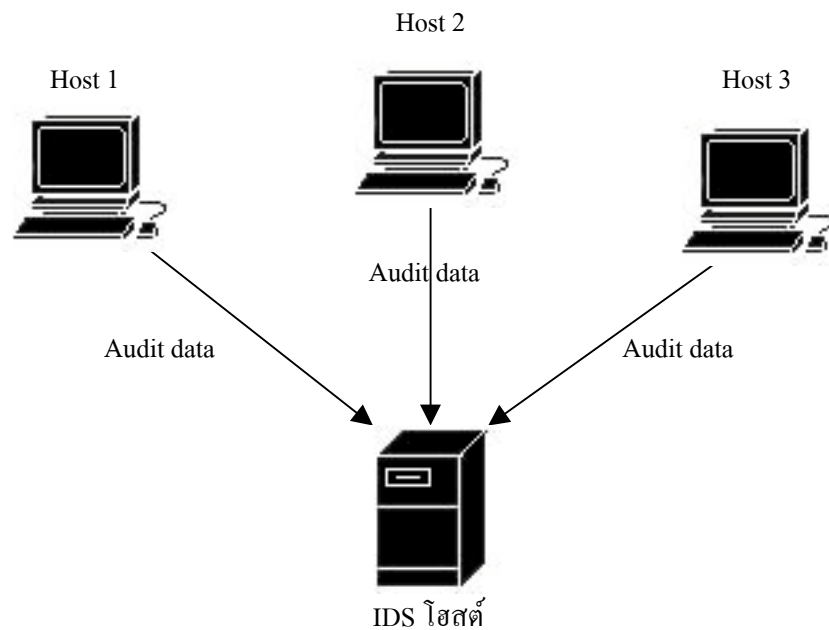


คู่มือการเขียนโปรแกรม



ภาพรวมการทำงานของระบบตรวจจับผู้บุกรุก

ภายในไดเรกทอรี src ซึ่งบรรจุโค้ดโปรแกรม IsagIDS อยู่ นั้นจะประกอบด้วยไฟล์และไดเรกทอรีดังนี้

1. **ไฟล์ config.c** ทำหน้าที่เก็บข้อมูลที่ต้องการในขณะทำการคอมไพล์ไฟล์และนำข้อมูลที่ต้องการเก็บไว้ในไฟล์ config.h
2. **ไฟล์ config.h** ถูกสร้างจากโปรแกรมที่คอมไพล์จาก config.c จะเก็บข้อมูลค่าคงที่ที่ใช้ใน audit module ดังนี้

WORKING_DIR ไดเรกทอรีที่โมดูลทำงานอยู่

CONFIG_FILE ตำแหน่งของไฟล์การตั้งค่าการเริ่มใช้งานของ audit module

TEMP_DATA ไฟล์ที่ใช้เก็บข้อมูลพฤติกรรมชั่วคราวในกรณีที่ไม่สามารถติดต่อ IDS โฮสต์ได้

MODULE_FSUID ค่า uid ที่ใช้ในการทำงานของโมดูล เช่นเขียนไฟล์ TEMP_DATA เป็นต้น

MODULE_FSGID ค่า gid ที่ใช้ในการทำงานของโมดูล เช่นเขียนไฟล์ TEMP_DATA เป็นต้น

3. **ไฟล์ define.h** ทำการนิยามค่าคงที่และโครงสร้างข้อมูลอื่นๆ

4. **ไดเรกทอรี auditModule** บรรจุโค้ดโปรแกรมทางฝั่ง audit module ของระบบ ประกอบด้วยไฟล์ดังนี้

configModule.c ทำหน้าที่ในการตั้งค่าหรือเปลี่ยนแปลงค่าการเริ่มต้นการใช้งานของ audit module

log.c โค้ดของ audit module

5. ได้เรททอรี server บรรจุโค้ดโปรแกรมทางฝั่ง IDS โฮสต์ โดยมีไฟล์สำคัญที่เกี่ยวกับ IDS เซิร์ฟเวอร์ ดังนี้

mainserver.c	เป็นไฟล์ที่มีฟังก์ชัน main ของโปรแกรมฝั่ง IDS เซิร์ฟเวอร์ (IDS เดมอน)
analyse.c,filter.c	เป็นส่วนของการวิเคราะห์พฤติกรรมผู้ใช้งาน
response.c	เป็นส่วนของการตอบสนองการวิเคราะห์
makemd5.c,md5c.c	ส่วนที่ใช้ในการตรวจสอบความถูกต้องของข้อมูลต่างๆ ที่เก็บไว้ ไฟล์นอกเหนือจากนั้นเป็นเครื่องมือต่างๆ ที่ได้เขียนเพิ่มเติมดังนี้
log2cvs.c	ทำการเปลี่ยนล็อกของพฤติกรรมผู้ใช้งานให้อยู่ในรูปแบบไฟล์ CVS ซึ่ง สามารถเปิดโดยโปรแกรมจำพวก spread sheet ได้
viewBlack.c	ทำการแสดงข้อมูลในไฟล์ blacklist

รายละเอียดย่อยในไฟล์ที่สำคัญ

ส่วน audit module

ไฟล์ log.c ฟังก์ชันที่สำคัญในไฟล์นี้มีดังนี้

1. *asmlinkage int new_execve(struct pt_regs)*

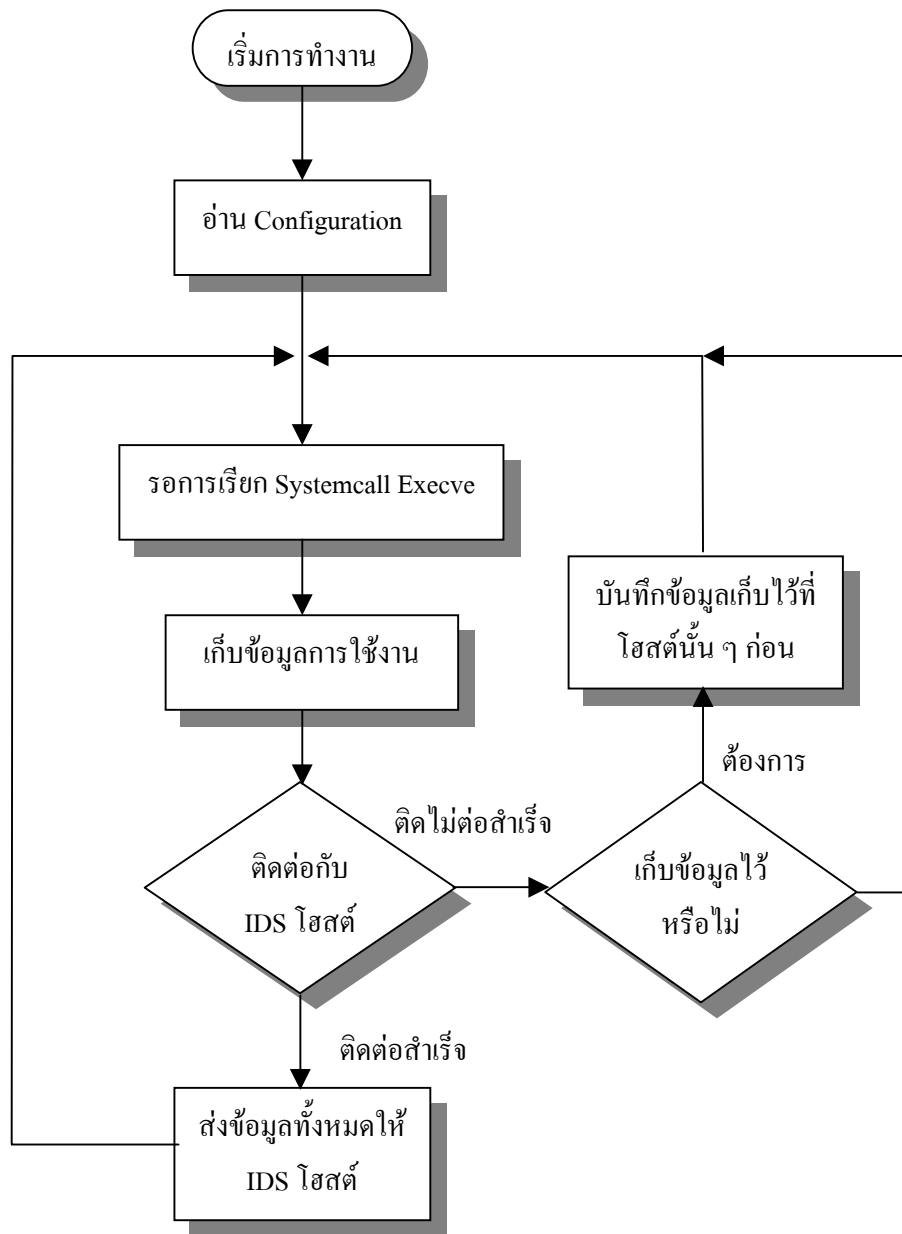
เป็นส่วนที่จัดการการทำงานของ system call execve ซึ่งถือได้ว่าเป็นฟังก์ชันการทำงานหลักในไฟล์นี้ โดยข้อมูลที่เราต้องการถูกเก็บไว้ตามรีจิสเตอร์ดังนี้

- ebx เก็บชื่อไฟล์ที่ทำการรัน
- ecx ชี้ไปยังอาร์กิวเมนต์ที่ส่งเข้ามา

ส่วนข้อมูลเวลาที่ทำการรันนั้นเราได้มาจากโคร CURRENT_TIME ซึ่งจะได้อ่านในหน่วยวินาทีเลย

ภายในฟังก์ชันนี้ยังมีการเปลี่ยน fsuid และ fsgid ให้เป็นไปตามค่าที่ถูกตั้งไว้ในค่า MODULE_FSUID และ MODULE_FSGID ตามลำดับซึ่งจะถูกเซตในช่วงการคอมไพล์โปรแกรมตามค่า uid และ gid ของผู้คอมไพล์โปรแกรม โดยค่าดังกล่าวจะใช้ในกรณีที่ audit module ไม่สามารถติดต่อกับ IDS โฮสต์และต้องทำการเก็บข้อมูลไว้ที่โฮสต์นั้นๆ ก่อนซึ่งข้อมูลที่เก็บนั้นจะถูกเก็บรวมกันอยู่ไฟล์เดียวที่ชื่อ tempdata โดยบันทึกเป็นเรคคอร์ดต่อกันไปและไฟล์นี้จะมีค่าเพอร์มิชชันเป็น rw- - - - - ซึ่งจะมีผู้ที่อ่านและเขียนได้เพียงคนเดียว(ถ้าไม่นับรูท) เราจึงต้องมียูสเซอร์คนกลางที่ทำหน้าที่เขียนไฟล์นี้ซึ่งก็คือยูสเซอร์ที่มีค่า uid และ gid ตามค่าที่เซตไว้ใน MODULE_FSUID และ MODULE_FSGID ตามลำดับ

เมื่อทำการเก็บข้อมูลที่ต้องการเสร็จเรียบร้อยแล้วจะส่งข้อมูลที่เก็บได้ไปยัง IDS โฮสต์โดยฟังก์ชัน `int send_to_socket(struct execInfor):` ซึ่งหากไม่สามารถส่งข้อมูลไปได้ก็จะทำการเก็บข้อมูลไว้ในไฟล์ tempdata ก่อน(หากเราได้ทำการ enable อปชันนี้ในตอนที่เราทำการ config ตัว audit module) หลังจากนั้นก็จะเซตค่า fsuid และ fsgid กลับเป็นค่าเดิม



ขั้นตอนการทำงานของ audit module

2. int send_to_socket(struct execInfor)

เป็นส่วนที่ทำหน้าที่ส่งข้อมูลพฤติกรรมไปให้ทาง IDS โดยเรียกใช้งาน socket โดยผ่านทาง system call socket call โฮสต์ซึ่งก่อนส่งข้อมูลจะมีการเข้ารหัสอย่างง่าย ๆ ไปด้วยเพื่อป้องกันผู้บุกรุกที่ทำการ sniff และอาจสังเกตเห็นโดยการเข้ารหัสนั้นเราทำโดยนำข้อมูลทั้งหมดมาต่อกับเลข 42 ดังรูป

ข้อมูลที่เก็บมาจาก System call execve	42 (1 ไบต์)
---------------------------------------	-------------

แล้วหลังจากนั้นจะนำข้อมูลทุกตัวไปบวกกับ 3 และทำการส่งข้อมูลที่ได้ไปให้ทางฝั่ง IDS โฮสต์ เหตุผลที่ทำการเข้ารหัสอย่างง่าย ๆ เนื่องมาจากทุกครั้งที่มีการรันโปรแกรมก็ต้องมีการส่งข้อมูลไปให้ IDS โฮสต์ทุกครั้ง ดังนั้นถ้าทำการเข้ารหัสอย่างซับซ้อนก็จะทำให้การทำงานของระบบช้าลง

ถ้าการส่งไม่มีข้อผิดพลาดก็จะส่งค่า 1 กลับมา

3. int save_audit_data(struct execInfor)

ทำหน้าที่เก็บข้อมูลลงไฟล์ก่อนในกรณีที่ยังติดต่อ IDS โฮสต์ไม่ได้โดยเขียนลงในไฟล์ที่ชื่อ tempdata ซึ่งตำแหน่งที่ทำการเก็บไฟล์นี้จะถูกตรวจสอบขณะคอมไพล์โปรแกรมโดยจะถูกเก็บอยู่ที่ TEMP_DATA ที่ config.h

ส่วน IDS โฮสต์

ไฟล์ mainserver.c ฟังก์ชันที่สำคัญในไฟล์นี้มีดังนี้

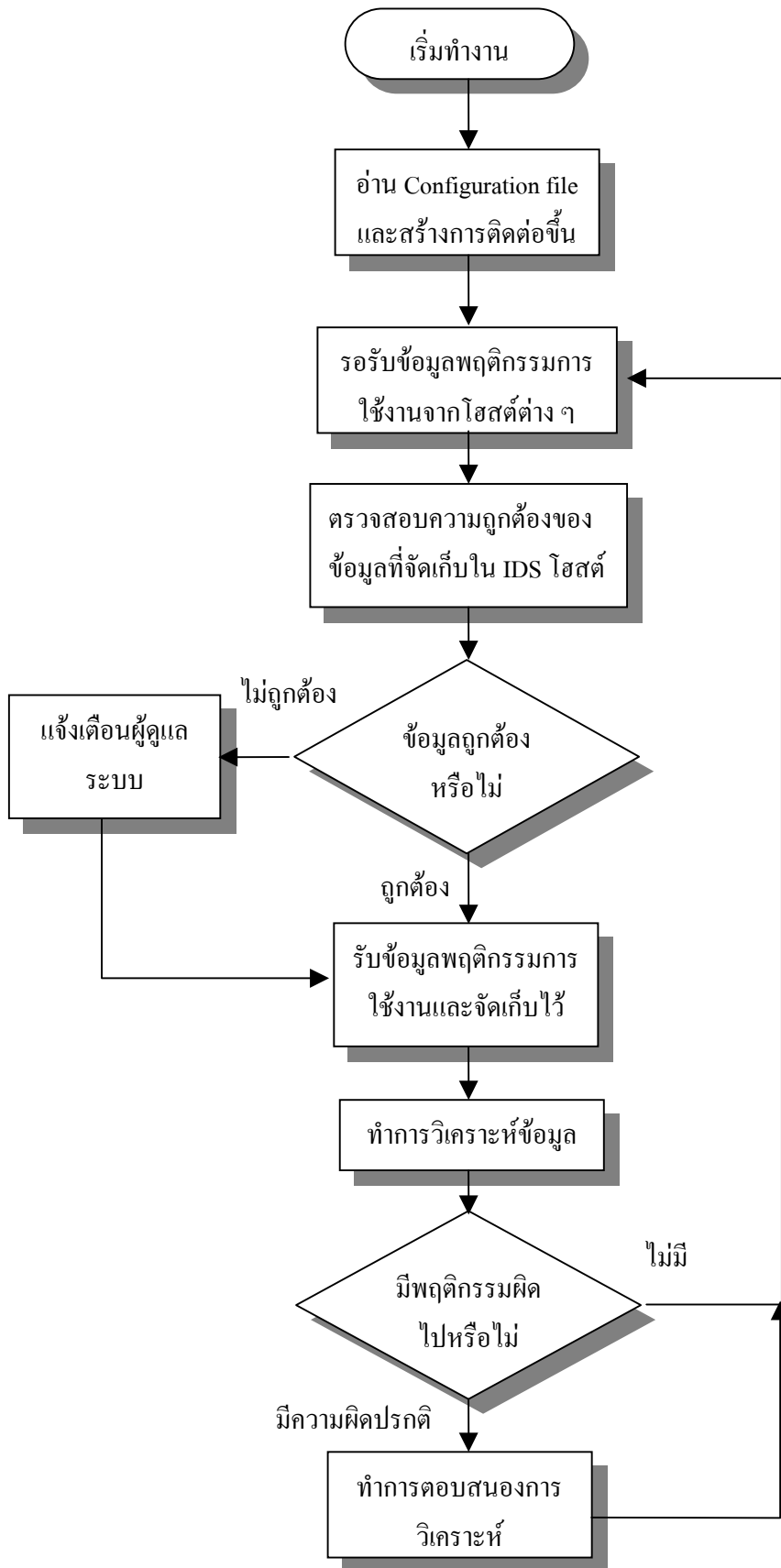
1. int main(int argc, char *argv[])

ส่วนฟังก์ชันหลักของการทำงานทั้งหมดของ IDS เดมอนซึ่งจะทำการเปลี่ยนให้โปรแกรมรันเป็นเดมอนโดยการเรียกฟังก์ชัน int startDaemon() และสร้างการติดต่อและรอรับข้อมูลที่ส่งมาจากทาง audit module เมื่อมีข้อมูลส่งมาก็จะ fork ตัวเองและนำข้อมูลที่ได้รับมาถอดรหัสและนำเอาไปด้สุดท้าย(ที่เราเพิ่มเลข 42 เข้าไปในขณะที่ทำการส่งข้อมูลมาจาก audit module) มาเทียบดูซึ่งหากเป็นเลข 42 ก็แสดงว่าเป็นข้อมูลที่ถูกต้องก็จะนำข้อมูลไปเข้าขั้นตอนการวิเคราะห์โค่นผ่านฟังก์ชัน float analyseData(struct sockaddr_in , struct execInfor) ซึ่งจะได้อ่านค่าเปอร์เซ็นต์คะแนนพฤติกรรมกลับมา หากตรวจพบพฤติกรรมที่ผิดปกติ(มีคะแนนต่ำกว่าค่า BEHAVIOUR_THRESHOLD) ก็จะไปเรียกทำส่วนตอบสนองการวิเคราะห์ผ่านฟังก์ชัน int responseIntrusion(struct execInfor ,struct sockaddr_in ,float *) แต่ถ้าหากพฤติกรรมไม่ผิดปกติก็จะตรวจสอบที่ blacklist ว่ามีชื่อผู้ใช้นี้อยู่หรือไม่ ถ้ามีก็จะทำการลบออกจาก blacklist และบันทึกเหตุการณ์ว่าได้มีการลบชื่อจาก blacklist ที่ eventlist ด้วย

2. float analyseData(struct sockaddr_in, struct execInfor)

เป็นส่วนที่ทำการตรวจสอบความถูกต้องของข้อมูลพฤติกรรมของแต่ละคน นำข้อมูลของแต่ละคนมาเก็บไว้ และทำการวิเคราะห์ข้อมูลพฤติกรรม

ในส่วนการตรวจสอบความถูกต้องของข้อมูลพฤติกรรมและนำข้อมูลของแต่ละคนไปเก็บนั้น (ไม่รวมช่วงการวิเคราะห์ข้อมูล) เป็นช่วงที่เราจะยอมให้มีการทำงานได้เพียงโปรเซสเดียวในขณะใดขณะหนึ่ง(หมายความว่าขณะที่มีโปรเซสหนึ่งกำลังทำการตรวจสอบความถูกต้องของข้อมูลและนำข้อมูลของแต่ละคนไปเก็บนั้น โปรเซสอื่นๆจะเข้ามาทำการตรวจสอบความถูกต้องและเก็บข้อมูลของผู้ใช้นั้นๆ ไม่ได้แต่สามารถอ่านข้อมูลของผู้ใช้นั้นไปทำการวิเคราะห์ได้ ทั้งนี้เพื่อป้องกันปัญหาที่เกิดจากการ update ฐานข้อมูลพฤติกรรมซ้อนกันโดยส่วน message digest ที่ใช้ในการตรวจสอบความถูกต้องของข้อมูลที่ทำในโปรเซสก่อนหน้านั้นยังสร้างไม่เสร็จซึ่งจะทำให้เกิดความผิดพลาดในการตรวจสอบความถูกต้อง



ขั้นตอนการทำงานของ IDS เดมอน

ต้องของข้อมูลได้) โดยการตรวจสอบความถูกต้องของข้อมูลนั้นเราได้นำเอาฐานข้อมูลของแต่ละคนไปผ่านอัลกอริทึม MD5 ซึ่งจะให้ผลลัพธ์ออกมาเป็นจำนวน 128 บิตและนำไปเก็บไว้ในไฟล์เช่นหากผู้ใช้งานมีหมายเลข uid เป็น '501' ไฟล์ที่เก็บผลลัพธ์หลังจากผ่านอัลกอริทึม MD5 ก็จะถูกเก็บไว้ในชื่อ '.501' ซึ่งต้องใช้คำสั่ง 'ls -a' ถึงจะเห็นไฟล์ดังกล่าว

หลังจากเก็บข้อมูลเรียบร้อยแล้วก็จะนำข้อมูลทั้งหมดไปเข้าขั้นตอนการวิเคราะห์ข้อมูลโดยมี 2 ขั้นตอนตามที่ได้อธิบายไว้ในปริญญานิพนธ์คือ

1. ช่วงเตรียมข้อมูลที่ใช้ในการวิเคราะห์ข้อมูล

ทำการแยกข้อมูลออกเป็น 2 ช่วงไว้ในไฟล์ชั่วคราวผ่านฟังก์ชัน `int readLog(char *,char *,char *)` ในไฟล์ `filter.c` ซึ่งจะได้ออกมา 2 ไฟล์คือไฟล์ฐานข้อมูลในช่วงระยะเวลาปัจจุบันและฐานข้อมูลในอดีต

2. ทำการคำนวณค่าความเหมือนของข้อมูลทั้ง 2 ชุด

นำไฟล์ทั้ง 2 ผ่านขั้นตอนการวิเคราะห์โดยผ่านฟังก์ชัน `int analyse(char *, char *)` ในไฟล์ `analyse.c` ค่าที่ส่งกลับจะเป็นค่าคะแนนความเหมือนที่ได้(คะแนนยังไม่ถูกคิดเป็นเปอร์เซ็นต์)

หลังจากผ่านขั้นตอนทั้ง 2 ก็จะนำเอาคะแนนมาหาค่าเป็นเปอร์เซ็นต์แล้วส่งค่ากลับไป

3. `int createConfigFile(int)`

ทำการสร้างไฟล์ที่ใช้เก็บค่าตัวแปรที่ใช้ในการวิเคราะห์แต่ละครั้งซึ่งจะถูกเรียกทุกครั้งที่มีโฮสต์ใหม่ส่งข้อมูลเข้ามาในระบบโดยจะทำการกำหนดค่า default ไว้ให้ที่ไฟล์ `config.serv` ซึ่งเราสามารถเปลี่ยนแปลงค่าตัวแปรในไฟล์นี้ได้ตลอดเวลา

ไฟล์ `filter.c`

```
int readLog(char *,char *,char *)
```

ทำการแยกฐานข้อมูลทั้งหมดออกเป็น 2 ส่วนคือฐานข้อมูลการใช้งานในช่วงปัจจุบันและช่วงอดีตเพื่อนำไปใช้งานต่อในส่วนการวิเคราะห์ข้อมูลโดยจำนวนข้อมูลนั้นจะถูกกำหนดตามตัวแปร `R_WINDOW_SIZE`(สำหรับจำนวนฐานข้อมูลช่วงปัจจุบันที่นำมาเปรียบเทียบ) และ `DB_WINDOW_SIZE`(สำหรับจำนวนฐานข้อมูลในช่วงอดีตที่นำมาเปรียบเทียบ)

ไฟล์ `analyse.c`

1. `int analyse(char *, char *)`

เป็นฟังก์ชันหลักของไฟล์นี้ทำการวิเคราะห์ฐานข้อมูลในช่วงปัจจุบันกับฐานข้อมูลในอดีตโดยใช้วิธีการตามที่ได้กล่าวไว้ในปริญญานิพนธ์ โดยค่าที่ส่งกลับมาจะเป็นค่าความเหมือนของไฟล์ทั้งสองไฟล์(ซึ่งจะบรรจุฐานข้อมูลในช่วงปัจจุบันและฐานข้อมูลช่วงอดีตอยู่)โดยในการคำนวณความเหมือนของแต่ละคู่จะทำผ่านฟังก์ชัน `int findSim(char [ ][],char [ ][])`

2. *int findSim(char [][],char [][])*

ทำการเปรียบเทียบค่าความเหมือนของข้อมูลในอาร์เรย์ทั้งสองโดยใช้อัลกอริทึมเปรียบเทียบความเหมือนดังที่กล่าวไว้ในปัญญานิพนธ์แล้วเช่นกัน โดยค่าที่ส่งกลับมาจะเป็นค่าความเหมือนของลำดับทั้งสอง

ไฟล์ response.c

1. *int responseIntrusion(struct execInfor,struct sockaddr_in,float *)*

เป็นฟังก์ชันหลักของไฟล์นี้ โดยการทำงานก็จะแสดงข้อความออกทางหน้าจอว่าตรวจพบพฤติกรรมที่ผิดปกติและแสดงข้อมูลต่างๆด้วย และทำการตรวจสอบว่าผู้ใช้นี้มีรายชื่ออยู่ใน blacklist แล้วหรือยังโดยผ่านฟังก์ชัน *int findBlackList(int uid,char *hostIP)* ถ้ายังไม่มีก็จะทำการเพิ่มรายชื่อลงไปใน blacklist ผ่านฟังก์ชัน *int writeBlackList(struct execInfor,char *,float *)* และบันทึกเหตุการณ์ไว้ใน eventlist ด้วยและทำการเมลล์แจ้งเตือนผู้ดูแลระบบ

2. *void mailToRoot(char *mesg)*

ทำการส่งเมลล์ไปให้ root

3. *int findBlackList(int uid,char *hostIP)*

ค้นหาว่ามีผู้ใช้นี้อยู่ใน blacklist หรือไม่หากพบจะส่งค่า 1 กลับมา

4. *int writeBlackList(struct execInfor,char *,float *)*

บันทึกผู้ใช้งานใน blacklist โดยก่อนที่จะทำการบันทึกข้อมูลนั้นจะมีการตรวจสอบความถูกต้องของไฟล์ blacklist ก่อนว่าถูกทำการเปลี่ยนแปลงหรือไม่ (การตรวจสอบความถูกต้องใช้อัลกอริทึม MD5 แบบเดียวกับที่ทำการตรวจสอบความถูกต้องของข้อมูลพฤติกรรม) หากพบความผิดปกติเช่นไฟล์อาจถูกลบหรือถูกเปลี่ยนแปลงก็จะบันทึกเหตุการณ์ไว้ใน eventlist ผ่านฟังก์ชัน *int writeEventList(struct execInfor,char *,float *,char *)*

เมื่อตรวจสอบความถูกต้องเสร็จก็ทำการบันทึกข้อมูลเพิ่มลงไปและทำการสร้าง message digest ขึ้นมาใหม่และบันทึกไว้ในไฟล์ '.blacklist'

5. *int removeBlackList(struct execInfor,struct sockaddr_in)*

ลบผู้ใช้นี้ออกจาก blacklist โดยจะทำการสร้างไฟล์ชั่วคราวขึ้นมากและทำการอ่านข้อมูลจาก blacklist ไปเรื่อยๆ ซึ่งหากข้อมูลที่เราอ่านได้ไม่ใช่ข้อมูลที่เราจะทำการลบก็จะนำไปบันทึกลงในไฟล์ชั่วคราวที่เราทำการสร้างขึ้นแต่หากพบข้อมูลที่ต้องการลบก็จะไม่นำมาบันทึกไว้ในไฟล์ชั่วคราวเมื่อทำการ

อ่านจนหมด blacklist ก็จะนำไฟล์ชั่วคราวที่สร้างขึ้นมาให้กลายเป็น blacklist ไฟล์ใหม่ และทำการหา message digest ของข้อมูลนี้ใหม่และบันทึกไว้ในไฟล์ '.blacklist'

6. *int writeEventList(struct execInfor, char *, float *, char *)*

บันทึกข้อมูลเหตุการณ์ต่างๆลงใน eventlist โดยก่อนที่จะทำการบันทึกข้อมูลนั้นก็จะทำการตรวจสอบความถูกต้องของข้อมูลในไฟล์ก่อนเช่นกันเหมือนกับไฟล์ blacklist แต่หากตรวจพบความผิดปกติในไฟล์ก็จะบันทึกข้อมูลลงในตัว eventlist เอง

